Conversione da decimale alle diverse basi

Unità di Apprendimento 1 · Lezione 4 (parte B)

La rappresentazione delle informazioni

Tecnologie e Progettazione di Sistemi Informatici e di Telecomunicazioni (TPSI)

Classe 3ª – Istituto Tecnico Informatico

Di cosa parliamo oggi

1. **Il problema inverso**: da decimale a binario, ottale, esadecimale
2. **L'algoritmo della divisione ripetuta** (numeri interi)
3. Esempi in **base 2**, **base 8**, **base 16**
4. **Numeri frazionari**: l'algoritmo della moltiplicazione ripetuta
5. Numeri **periodici** e precisione

> Obiettivo: saper convertire qualsiasi numero decimale (intero o con la virgola) in un'altra base.

Ripasso: dalla base *b* al decimale

Nei sistemi di numerazione posizionali, data una cifra in **una qualunque base** è possibile determinarne direttamente il **valore decimale** con una semplice operazione di **addizione**.

$$\begin{array}{cccccccc} 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0_2 \\ 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ & \uparrow & & & \uparrow & & \uparrow & \end{array}$$

$$2^6 + 2^3 + 2^1 = 64 + 8 + 2 = 74_{10}$$

(BASE 2) (BASE 10)

Somma dei **pesi** delle cifre diverse da zero. Oggi facciamo il percorso **opposto**.

1. Da decimale intero a un'altra base

Il problema opposto

Non è così immediato il problema opposto: passare da un numero espresso in **base decimale** al numero espresso nelle diverse basi:

- **binaria**
- **ottale**
- **esadecimale**

Si applica l'**algoritmo della divisione ripetuta**.

Perché funziona? Dividere per la base *b* "stacca" l'ultima cifra del numero nella nuova base: il **resto** è la cifra meno significativa, il **quoziente** contiene tutte le altre. Ripetendo, si estraggono le cifre una alla volta.

L'algoritmo della divisione

1. Se il numero da convertire è **0** → non c'è altro da convertire; altrimenti:
   - **A.** si divide il numero per il **divisore** (base di destinazione) e si individua il **resto**
   - **B.** si sostituisce il valore con il **quoziente** della divisione
   - **C.** si torna al passo **1** e si ricomincia la divisione
2. Si leggono i **resti** nell'ordine **opposto** a quello con cui sono stati trovati.

> Questo algoritmo vale per la conversione dalla base decimale a **tutte le altre basi**: cambia solo il **divisore**, che è sempre la **base di destinazione** (2, 8 o 16).

2. Da decimale a binario

Esempio 1: 59_{10} in binario (passo 1)

Convertiamo $N = 59_{10}$. Dividiamo 59 per 2, ottenendo quoziente 29 e resto 1:

```
   Divisore    Quoziente    Resto
       ↓           ↓          ↓
      59   :   2   =   29   +   1
```

Mettiamo i valori in una tabella:

Passo	Valore	Divisore	Quoziente	Resto
1	59	2	29	1

Esempio 1: 59_{10} in binario (passo 2)

Sostituiamo al valore 59 il **quoziente 29** e ripetiamo la divisione per 2:

```
29 : 2 = 14 + 1
```

Aggiungiamo una riga alla tabella:

Passo	Valore	Divisore	Quoziente	Resto
1	59	2	**29**	1
2	**29** ←	2	14	1

Il quoziente di una riga diventa il **valore** della riga successiva.

Esempio 1: si continua fino a quoziente 0

Continuiamo a ripetere il procedimento **finché il quoziente diventa 0**:

Iterazione	Dividendo	Divisore	Quoziente	Resto
1	59	2	29	1
2	29	2	14	1
3	14	2	7	0
4	7	2	3	1
5	3	2	1	1
6	1	2	**0**	1

Esempio 1: lettura del risultato

Alla **sesta iterazione** il quoziente vale 0: l'algoritmo **termina**.

Leggiamo i resti **«a rovescio»**, dall'ultimo al primo:

1 1 1 0 1 1

> $59_{10} = 111011_2$

Verifica (conversione inversa): 32 + 16 + 8 + 0 + 2 + 1 = **59** ✓

Esempio 2: 43_{10} in binario

```
43 : 2 = 21 + 1     ← LSB (bit meno significativo)
21 : 2 = 10 + 1
10 : 2 =  5 + 0
 5 : 2 =  2 + 1
 2 : 2 =  1 + 0
 1 : 2 =  0 + 1     ← MSB (bit più significativo)
```

I resti, nell'ordine in cui li troviamo, sono: **1 1 0 1 0 1**.

«Rigirandoli» si ottiene **101011**, cioè il valore binario di 43:

> $43_{10} = 101011_2$

Verifica: 32 + 8 + 2 + 1 = 43 ✓

Il metodo dei pesi (alternativa mentale)

Per numeri piccoli si può evitare la tabella usando i **pesi** 128 · 64 · 32 · 16 · 8 · 4 · 2 · 1.

Esempio: 43 → si parte dal peso più grande che ci sta:

128	64	32	16	8	4	2	1	Resto
0	0	**1**	0	**1**	0	**1**	**1**	

43 − 32 = 11 → 11 − 8 = 3 → 3 − 2 = 1 → 1 − 1 = 0

Risultato: 00101011_2 (con 8 bit) = 101011_2 (senza zeri iniziali).

3. Da decimale a ottale

Divisore 8: 3157_{10} in base 8 (passo 1)

L'algoritmo è **il medesimo**, ma ora il divisore è **8**.

Convertiamo 3157_{10} in base 8:

```
   Divisore    Quoziente    Resto
       ↓           ↓          ↓
     3157  :  8  =  394   +   5
```

Iterazione	Dividendo	Divisore	Quoziente	Resto
1	3157	8	394	5

3157_{10} in base 8: la tabella completa

Ripetiamo il procedimento fino a ottenere la seguente tabella:

Iterazione	Dividendo	Divisore	Quoziente	Resto
1	3157	8	394	5
2	394	8	49	2
3	49	8	6	1
4	6	8	**0**	6

Ottenuto **0** come quoziente la divisione termina.

Resti dall'**ultimo al primo** (dal basso verso l'alto): $N_8 = 6\ 1\ 2\ 5$

> $3157_{10} = 6125_8$

Verifica: $6 \cdot 512 + 1 \cdot 64 + 2 \cdot 8 + 5 = 3072 + 64 + 16 + 5 = 3157$ ✓

Esempio: 7043_{10} in base 8

```
7043 : 8 = 880 + 3
 880 : 8 = 110 + 0
 110 : 8 =  13 + 6
  13 : 8 =   1 + 5
   1 : 8 =   0 + 1
```

Resti letti «a rovescio»: **1 5 6 0 3**

> $7043_{10} = 15603_{8}$

Verifica: 1·4096 + 5·512 + 6·64 + 0·8 + 3 = 4096 + 2560 + 384 + 3 = 7043 ✓

4. Da decimale a esadecimale

Divisore 16: 3157_{10} in base 16

L'algoritmo è **il medesimo**, ma ora il divisore è **16**.

Convertiamo 3157_{10} in base 16:

```
   Divisore     Quoziente     Resto
       ↓            ↓           ↓
     3157   :   16   =   197   +   5
```

Iterazione	Dividendo	Divisore	Quoziente	Resto
1	3157	16	197	5
2	197	16	12	5
3	12	16	0	C

3157_{10} in base 16: il risultato

Ottenuto **0** come quoziente la divisione termina.

Resti dall'ultimo al primo: N_{16} = **C 5 5**

$3157_{10} = C55_{16}$

Attenzione ai resti ≥ 10: vanno scritti con la lettera corrispondente. Qui il resto **12** diventa **C**.

Verifica: 12·256 + 5·16 + 5 = 3072 + 80 + 5 = 3157 ✓

Esempio: 44157_{10} in base 16

```
44157 : 16 = 2759 + 13  (D)
 2759 : 16 =  172 +  7
  172 : 16 =   10 + 12  (C)
   10 : 16 =    0 + 10  (A)
```

Resti dall'ultimo al primo: **A C 7 D**

> $44157_{10} = AC7D_{16}$

Ricordiamo che nel sistema esadecimale: **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**.

Verifica: 10·4096 + 12·256 + 7·16 + 13 = 40960 + 3072 + 112 + 13 = 44157 ✓

Riepilogo: numeri interi

Base di arrivo	Divisore	Esempio	Risultato
2 (binario)	2	59_{10}	111011_2
2 (binario)	2	43_{10}	101011_2
8 (ottale)	8	3157_{10}	6125_8
8 (ottale)	8	7043_{10}	15603_8
16 (esadecimale)	16	3157_{10}	$C55_{16}$
16 (esadecimale)	16	44157_{10}	$AC7D_{16}$

Regola: dividere per la base di arrivo, **fermarsi a quoziente 0**, leggere i resti **dal basso verso l'alto**.

Esercizi sugli interi

Converti dal decimale:

Decimale	Binario	Ottale	Esadecimale
37	?		
100	?	?	?
200	?		
500		?	
1000			?

Soluzioni: 37 = 100101_2 · 100 = 1100100_2 = 144_8 = 64_{16} · 200 = 11001000_2 · 500 = 764_8 · 1000 = $3E8_{16}$

5. Da decimale frazionario alle diverse basi

Numeri frazionari: la regola

La **parte frazionaria** del numero in base *b* si ottiene **moltiplicando ripetutamente** la parte frazionaria del numero decimale per la base *b* e registrando la **parte intera** dei successivi prodotti, presi **dall'alto verso il basso**.

> I numeri frazionari che hanno una rappresentazione **esatta** in base *b* daranno a un certo punto un risultato uguale a **0**. Altrimenti ci si ferma alla **precisione desiderata**.

Attenzione: la parte **intera** si converte con la **divisione** (lettura dal basso), la parte **frazionaria** con la **moltiplicazione** (lettura dall'alto).

L'algoritmo della moltiplicazione

Passo 1) si stabilisce la **precisione** desiderata (es. 3 bit dopo la virgola)

Passo 2) si **moltiplica** la parte decimale per la base:

- se si ottiene **0** → termine conversione
- altrimenti:
  - **A.** si evidenzia la **parte intera** del risultato (cioè se si supera o meno l'unità)
  - **B.** si sostituisce al numero la **parte decimale** del prodotto ottenuto
  - **C.** si torna al passo B e si ripete la moltiplicazione

Passo 3) si leggono i numeri segnati **nell'ordine** in cui sono stati trovati.

Condizione di terminazione: si ottiene 0 **oppure** si è raggiunto il numero di cifre stabilito (il numero potrebbe essere **periodico**).

Esempio completo: 124.125_{10} in binario

Parte intera: 124 → divisioni successive per 2:

```
124 : 2 = 62 + 0     ← LSB
 62 : 2 = 31 + 0
 31 : 2 = 15 + 1
 15 : 2 =  7 + 1
  7 : 2 =  3 + 1
  3 : 2 =  1 + 1
  1 : 2 =  0 + 1     ← MSB
```

Resti dal basso: **1111100** → $124_{10} = 1111100_2$

Verifica: 64 + 32 + 16 + 8 + 4 = 124 ✓

124.125_{10}: la parte frazionaria (0.125)

Precisione: **3 bit** dopo la virgola → 3 iterazioni.

Iter.	Valore	Moltiplicatore	Risultato	Bit parte frazionaria
1	0.125	2	0.250	0
2	0.250	2	0.500	0
3	0.500	2	1.000	1

Alla terza iterazione otteniamo **0** nella parte decimale: **fine**.

$.125_{10} = .001_2$

> $124.125_{10} = 1111100.001_2$

Verifica: 1/8 = 0,125 ✓

Esempio 1: 0.21875_{10} (precisione 5 bit)

La parte intera è **0**, quindi non serve conversione.

Iterazione	Valore	Moltiplicatore	Risultato	Bit frazionario
1	0.21875	2	**0**.4375	**0**
2	0.4375	2	**0**.875	**0**
3	0.875	2	**1**.75	**1**
4	0.75	2	**1**.5	**1**
5	0.5	2	**1**.0	**1**

$0.21875_{10} = 0.00111_2$

Verifica: 1/8 + 1/16 + 1/32 = 0,125 + 0,0625 + 0,03125 = 0,21875 ✓

Esempio 2: 0.45_{10} in binario (precisione 7 bit)

Iterazione	Valore	Moltiplicatore	Risultato	Bit frazionario
1	0.45	2	0.9	0
2	0.9	2	1.8	1
3	0.8	2	1.6	1
4	0.6	2	1.2	1
5	0.2	2	0.4	0
6	0.4	2	0.8	0
7	0.8	2	1.6	1

$0.45_{10} = 0.0111001\ldots_2$ → in realtà è **periodico**: $0.011100110011001100110011\ldots_2$

Numeri periodici: perché succede

Il risultato dell'esempio precedente **non termina mai**: dopo l'iterazione 3 si ripete la sequenza di valori **0.8 → 0.6 → 0.2 → 0.4 → 0.8 …**

> **Non tutti i decimali "finiti" sono finiti in binario!**
> Solo le frazioni con denominatore potenza di 2 (1/2, 1/4, 1/8, …) hanno rappresentazione **esatta**.

Esempio famoso: $0.1_{10} = 0.0001100110011\ldots_2$ (periodico).

Per questo in molti linguaggi `0.1 + 0.2` non dà esattamente `0.3`: il computer memorizza solo un'**approssimazione**!

Da decimale frazionario a ottale

Esempio: 0.45_{10} in base 8 (moltiplicatore 8)

Iterazione	Valore	Moltiplicatore	Risultato	Cifra frazionaria
1	0.45	8	3.6	3
2	0.6	8	4.8	4
3	0.8	8	6.4	6
4	0.4	8	3.2	3
5	0.2	8	1.6	1
6	0.6	8	4.8	4

$0.45_{10} = 0.346314_{8}$ (approssimato, periodico)

Da decimale frazionario a esadecimale

Esempio: 0.45_{10} in base 16 (moltiplicatore **16**)

Iterazione	Valore	Moltiplicatore	Risultato	Cifra frazionaria
1	0.45	16	**7**.2	7
2	0.2	16	**3**.2	3
3	0.2	16	**3**.2	3
4	0.2	16	**3**.2	3

$0.45_{10} = 0.7333_{16}$ (approssimato: la cifra 3 si ripete all'infinito)

Il valore 0.2 ricompare a ogni passo → il numero è **periodico** (periodo: la cifra 3).

Confronto: 0.45_{10} nelle tre basi

Base	Rappresentazione	Cifre frazionarie usate
2	$0.0111001\ldots_2$	7
8	$0.346314\ldots_8$	6
16	$0.7333\ldots_{16}$	4

Come per gli interi: **più grande è la base, meno cifre servono** per la stessa precisione.

Esercizi sui numeri frazionari

Converti dal decimale (precisione a piacere, max 6 cifre frazionarie):

Decimale	Binario	Ottale	Esadecimale
0.625	?		
0.75	?		?
0.5		?	
6.25	?		

Soluzioni: 0.625 = 0.101_2 · 0.75 = 0.11_2 = $0.C_{16}$ · 0.5 = 0.4_8 · 6.25 = 110.01_2

Esempio svolto (0.625): 0.625·2 = 1.25 → 1 · 0.25·2 = 0.5 → 0 · 0.5·2 = 1.0 → 1 → 0.101_2

Procedura completa: numero reale

Per convertire un numero reale **N = parte intera . parte frazionaria** da decimale alla base *b*:

1. **Separa** la parte intera dalla parte frazionaria
2. **Parte intera:** divisioni per *b*, resti letti **dal basso verso l'alto**
3. **Parte frazionaria:** moltiplicazioni per *b*, parti intere lette **dall'alto verso il basso**
4. **Riunisci** i due risultati con la virgola: `intera . frazionaria`
5. **Verifica** riconvertendo il risultato in decimale

Errori comuni da evitare

1. **Leggere i resti dall'alto** invece che dal basso (nella parte intera)
2. **Leggere le parti intere dal basso** (nella parte frazionaria: si legge dall'alto!)
3. **Fermarsi troppo presto:** nella divisione, il quoziente deve arrivare a **0**
4. **Dimenticare la lettera** per resti 10–15 in esadecimale
5. Nella moltiplicazione, **moltiplicare l'intero risultato** invece della sola **parte decimale**
6. Non indicare la **base** del risultato e la **precisione** usata

Riepilogo

	Parte intera	Parte frazionaria
Operazione	divisione per la base	moltiplicazione per la base
Si tiene	il **resto**	la **parte intera** del prodotto
Si continua con	il quoziente	la parte decimale del prodotto
Si ferma quando	quoziente = 0	risultato = 0 o precisione raggiunta
Lettura	dal **basso** verso l'alto	dall'**alto** verso il basso

Numeri periodici → **approssimazione** con un numero finito di cifre.

Grazie per l'attenzione

Domande?